

ECLIPSE · SEGURANÇA

Relatório de Auditoria de Segurança

— Eclipse

DATA DA REVISÃO	14 de setembro de 2026
ESCOPO	Frontend Angular, API NestJS, TypeORM/PostgreSQL, autenticação por sessão, armazenamento MinIO/S3, Docker Compose, scripts, documentação, histórico Git e bundle de produção.
BASE	Estado do workspace auditado, incluindo alterações locais não commitadas presentes no momento da revisão.

Nota metodológica. As cinco categorias foram adaptadas à stack detectada: isolamento manual por ownerId/projectId; autorização de papéis cruzada entre Angular e NestJS; IDOR revisado em todos os handlers; segredos revisados em código, Compose, configs, scripts, documentação, histórico Git e bundle; XSS revisado em templates Angular, Markdown, URLs e saídas PDF.

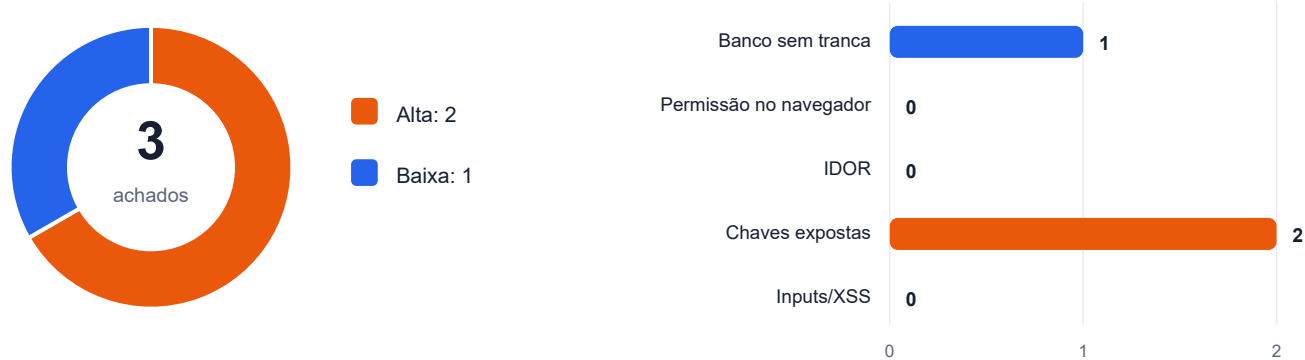
3 achados verificados · 2 altos · 1 baixo · 0 críticos

Este relatório evita hipóteses: condições necessárias à exploração são registradas junto de cada achado.

Resumo executivo

0 Críticos	2 Altos	0 Médios	1 Baixos
---------------	------------	-------------	-------------

A aplicação apresenta boa cobertura de autenticação e posse nos recursos. O risco central está fora da API: o Compose publica PostgreSQL e MinIO com credenciais previsíveis, de modo que a exposição de rede contorna todos os controles de aplicação. O segundo risco é um vazamento limitado de telemetria agregada no endpoint de privacidade.



Severidade considera impacto e pré-condições. F-02 e F-03 são altos porque permitem comprometimento integral quando as portas do host estão acessíveis; a condição de rede impede classificá-los como críticos no estado revisado.

Stack e mapeamento

Camada	Detectado	Mapeamento de segurança
Frontend	Angular 19 + Ionic 8, TypeScript	Gates de UI, [innerHTML], URLs, DOM e bundle
Backend	NestJS 11 / Express	Todos os decorators de rota, guards e serviços chamados
Persistência	TypeORM 1.1 + PostgreSQL 17/pgvector	Filtros ownerId/projectId, joins, agregações e SQL bruto
Autenticação	Sessão opaca em cookie HttpOnly; hash SHA-256 no banco	SessionAuthGuard e CurrentUser; não há papéis/admin
Armazenamento	S3 compatível / MinIO	Posse antes de URLs assinadas e exposição do serviço
Deploy	Docker Compose; sem CI/Helm/Terraform no repositório	Portas, imagens, defaults e credenciais

Pontos fortes e cobertura

Controle	Evidência	Resultado
Sessões protegidas	backend/src/auth/session-cookie.ts:5-11 e auth.service.ts:102-116	Cookie HttpOnly, SameSite=Lax e Secure em produção; token opaco de 256 bits e somente hash persistido.
Autenticação das rotas	backend/src/auth/session-auth.guard.ts:15-26	52 de 56 rotas exigem sessão. As quatro públicas são cadastro, login, logout idempotente e health.
Projetos e conversas	backend/src/projects/projects.service.ts:57-77,267-307	Listagens filtram ownerId; IDs de conversa são validados junto do projectId possuído.
Biblioteca privada	backend/src/library/library.service.ts:261-317,475-481	Listagem e busca filtram ownerId; playback, reprocessamento e remoção resolvem trackId + ownerId.
Referências e moodboards	backend/src/references/references.service.ts:89-141; moodboards.service.ts:139-206	Projeto é validado antes de ler/escrever; IDs filhos permanecem vinculados ao projectId.
Exportação LGPD	backend/src/privacy/privacy.service.ts:94-118	Todas as coleções exportadas aplicam owner_id direta ou indiretamente.
XSS/Markdown	src/app/pipes/markdown.pipe.ts:124-149; message.component.html:14	O renderer escapa &, <, >, aspas e apóstrofes antes de criar tags permitidas; Angular ainda sanitiza [innerHTML].
Validação global	backend/src/bootstrap.ts:25-34	ValidationPipe usa whitelist, rejeita campos extras e transforma DTOs; Helmet está ativo.
Segredos reais	histórico Git e dist/eclipse/browser	Nenhuma chave real foi encontrada no histórico; o bundle de produção não contém os padrões de segredo verificados.

Cobertura rota a rota

Foram enumerados os 56 handlers NestJS presentes em 13 controllers. A tabela resume todos os grupos; cada grupo corresponde à totalidade dos decorators @Get/@Post/@Put/@Patch/@Delete encontrados no arquivo, não a uma amostra.

Grupo	Rotas	Resultado
auth	5	register/login/logout públicos por desenho; me e disable protegidos pelo SessionAuthGuard
health	1	health público; somente status, serviço, timestamp e uptime
projects + conversations + messages	9	sessão e posse por ownerId; conversationId também vinculado ao projectId
AI streaming	1	sessão; criação/contexto passam por getActiveConversation(ownerId, projectId, conversationId)
briefings	5	sessão; projeto ativo/possuído validado antes de leitura, versão, escrita e confirmação
references	4	sessão; projeto validado e referenceId consultado junto com projectId
reference curation	6	sessão; projeto ativo, faixa por ownerId e IDs de referências vinculados ao projeto
library	7	sessão; listagem/busca por ownerId e operações por trackId + ownerId
moodboards	5	sessão; projeto possuído antes de versão, listagem, geração e PDF
final works	3	sessão; projectId + ownerId e trackId + ownerId + sourceProjectId
privacy	5	sessão; export/profile/consent/account isolados; usage tem F-01
evaluation	2	sessão; requireProject verifica projectId + ownerId
stem separation	3	sessão; projeto, referência e stem encadeados ao projeto do dono

Total: 56 rotas. Achado de cobertura: F-01 em privacy/usage. Nenhum IDOR foi confirmado nos demais handlers.

Achados detalhados

Severidade	Arquivo:linha	Descrição
BAIXA	backend/src/privacy/privacy.controller.ts:17-19 backend/src/privacy/privacy.service.ts:32-61	F-01 — Endpoint de privacidade expõe métricas globais de outros usuários O controller aceita qualquer sessão válida e passa ownerId, mas apenas a soma de tokens por mensagem usa esse parâmetro. Os contadores de YouTube, Cloudflare e Groq são lidos de tabelas diárias globais, sem owner_id. Assim, um usuário comum consegue consultar a atividade agregada de toda a instalação e inferir uso de outros usuários. Não há exposição de conteúdo ou identidade.
ALTA	backend/compose.yaml:6-11 backend/.env.example:4-8 backend/src/config/environment.config.ts:35-43 backend/evaluation/generate-report.mjs:7-13	F-02 — PostgreSQL é publicado com credencial previsível O Compose publica a porta 5432 em todas as interfaces do host e inicializa o banco com usuário e senha conhecidos do repositório. A validação aceita o mesmo default fora de produção e o script de avaliação repete o fallback. Um atacante com alcance de rede pode autenticar diretamente no PostgreSQL e ler ou alterar todos os dados, contornando SessionAuthGuard e filtros de ownerId.
ALTA	backend/compose.yaml:21-32 backend/.env.example:40-44 backend/src/config/environment.config.ts:91-103	F-03 — MinIO publica API e console com credencial root previsível A API e o console administrativo do MinIO são publicados em todas as interfaces com credenciais root conhecidas. Um atacante com alcance de rede pode listar, baixar, substituir ou excluir os áudios privados, sem passar pela emissão de URLs assinadas e sem checagem de posse.

F-01 · Endpoint de privacidade expõe métricas globais de outros usuários

BAIXA

Banco sem tranca

Arquivo(s): backend/src/privacy/privacy.controller.ts:17-19; backend/src/privacy/privacy.service.ts:32-61

```
async usage(ownerId: string) {
  ... WHERE p.owner_id = $1 ...
  SELECT ... FROM youtube_quota_usage WHERE usage_date = CURRENT_DATE
  SELECT ... FROM embedding_usage_daily WHERE usage_date = CURRENT_DATE
  SELECT ... FROM groq_usage_daily WHERE usage_date = CURRENT_DATE
}
```

Por que é explorável. O controller aceita qualquer sessão válida e passa ownerId, mas apenas a soma de tokens por mensagem usa esse parâmetro. Os contadores de YouTube, Cloudflare e Groq são lidos de tabelas diárias globais, sem owner_id. Assim, um usuário comum consegue consultar a atividade agregada de toda a instalação e inferir uso de outros usuários. Não há exposição de conteúdo ou identidade.

Condição. Explorável por qualquer conta autenticada quando a instalação possui mais de um usuário.

Correção indicada. Separar explicitamente métricas pessoais e operacionais. Para a resposta ao usuário, persistir/consultar contadores por owner_id; ou remover os totais globais. Se os totais globais forem necessários, movê-los para uma rota administrativa com autorização no backend.

F-02 · PostgreSQL é publicado com credencial previsível

ALTA

Chaves expostas

Arquivo(s): backend/compose.yaml:6-11; backend/.env.example:4-8; backend/src/config/environment.config.ts:35-43; backend/evaluation/generate-report.mjs:7-13

```
POSTGRES_USER: eclipse
POSTGRES_PASSWORD: eclipse_dev
ports:
  - "5432:5432"
DATABASE_PASSWORD ... default('eclipse_dev')
```

Por que é explorável. O Compose publica a porta 5432 em todas as interfaces do host e inicializa o banco com usuário e senha conhecidos do repositório. A validação aceita o mesmo default fora de produção e o script de avaliação repete o fallback. Um atacante com alcance de rede pode autenticar diretamente no PostgreSQL e ler ou alterar todos os dados, contornando SessionAuthGuard e filtros de ownerId.

Condição. Exige o Compose em execução e porta 5432 alcançável pela rede/firewall; o risco não depende da API estar exposta.

Correção indicada. Gerar senha local aleatória fora do repositório, exigir DATABASE_PASSWORD sem fallback e publicar a porta apenas em 127.0.0.1 quando o acesso externo não for necessário. Rejeitar credenciais conhecidas no startup, independentemente de NODE_ENV.

F-03 · MinIO publica API e console com credencial root previsível

ALTA

Chaves expostas

Arquivo(s): backend/compose.yaml:21-32; backend/.env.example:40-44;
backend/src/config/environment.config.ts:91-103

```
MINIO_ROOT_USER: eclipse_minio
MINIO_ROOT_PASSWORD: eclipse_minio_dev
ports:
- "9000:9000"
- "9001:9001"
```

Por que é explorável. A API e o console administrativo do MinIO são publicados em todas as interfaces com credenciais root conhecidas. Um atacante com alcance de rede pode listar, baixar, substituir ou excluir os áudios privados, sem passar pela emissão de URLs assinadas e sem checagem de posse.

Condição. Exige o Compose em execução e portas 9000/9001 alcançáveis pela rede/firewall.

Correção indicada. Usar credenciais aleatórias em arquivo local ignorado ou secret store, recusar os valores de exemplo no startup e vincular 9000/9001 a 127.0.0.1. Em cenários compartilhados, usar usuário de serviço sem privilégios root e políticas restritas ao bucket.

Resultado por categoria

1. Banco sem tranca

1 achado baixo

O mecanismo é filtro manual por ownerId/projectId, não RLS. Listagens, buscas, exportações e agregações foram revisadas. F-01 é o único desvio: três contadores globais em /privacy/usage. Exportação de dados e recursos de projeto estão corretamente isolados.

2. Permissão definida no navegador

Não se aplica

Não há role, isAdmin, canEdit, painel administrativo ou autorização por papel no frontend/backend. As ações de escrita disponíveis pertencem ao próprio usuário e são verificadas por sessão e posse no servidor; portanto não existe gate de papel de UI a cruzar.

3. IDOR

Nenhum achado

Todos os 56 handlers foram percorridos. Das 52 rotas autenticadas, as que recebem projectId, conversationId, referenceId, trackId ou stemId validam o recurso pai e/ou incluem ownerId/projectId no lookup. Rotas públicas não recebem IDs de objetos privados.

4. Chaves expostas

2 achados altos

Não foram encontradas chaves reais no código, histórico Git ou bundle Angular. Foram confirmadas credenciais padrão operacionais no Compose para PostgreSQL e MinIO, combinadas com portas publicadas em todas as interfaces (F-02 e F-03).

5. Inputs sem tratamento (XSS)

Nenhum achado

O único [innerHTML] usa o pipe Markdown próprio. O pipe escapa HTML antes de aplicar formatação e Angular sanitiza o valor; blocos de código também são escapados. URLs de templates passam pelo sanitizer do Angular, usam protocolos http/https validados e links externos possuem rel=noopener noreferrer. new Function existe apenas para importar módulos ESM por string constante, sem entrada do usuário.

Pontos fracos centrais

A fronteira de confiança mais frágil é o deploy local: banco e storage podem ficar acessíveis pela rede com credenciais públicas. Dentro da API, o único desvio confirmado é a ausência de separação entre métricas pessoais e globais no endpoint de privacidade.

Recomendações priorizadas

Prioridade	Ação	Resultado esperado
P1	Fechar a exposição direta de PostgreSQL e MinIO	Alterar o Compose para bind em 127.0.0.1, retirar credenciais fixas, gerar segredos locais e recusar valores conhecidos. Fazer isso antes de usar a máquina em rede compartilhada.
P2	Reduzir privilégios do armazenamento	Criar usuário/política MinIO exclusiva para o bucket e operações necessárias; a aplicação não deve usar credencial root.
P3	Separar telemetria pessoal de operacional	Decidir o contrato de /privacy/usage, persistir métricas pessoais por owner_id e proteger qualquer visão global por autorização administrativa no backend.
P4	Adicionar regressões de segurança	Criar E2E com duas contas para métricas de uso e teste de configuração que falhe para defaults públicos; manter os testes de posse já existentes.
P5	Automatizar detecção de segredos	Adicionar scanner de segredos no CI e varrer histórico. A auditoria atual não encontrou segredo real, mas a prevenção reduz risco futuro.

Verificações executadas

- Build NestJS: aprovado.
- Build Angular de produção: aprovado; bundle resultante sem padrões de chave/segredo.
- Testes unitários: 25 suítes, 111 testes aprovados.
- Testes E2E: 1 suíte, 23 testes aprovados.
- Histórico Git: patches e nomes de arquivos de ambiente revisados; nenhum segredo real confirmado.
- Deploy: Docker Compose revisado; não há arquivos CI, Helm ou Terraform no repositório.

ISSUES PARA O GITHUB

Os três achados acionáveis foram agrupados em duas issues: PostgreSQL e MinIO compartilham a mesma causa de deploy e foram consolidados para evitar spam.

Issue 1

```
--- ISSUE 1 ---
Título: [Segurança] Isolar métricas de uso por usuário no endpoint de privacidade
Labels sugeridas: security, severity:low

## Descrição do problema
O endpoint autenticado `GET /api/privacy/usage` recebe o usuário atual, porém retorna contadores globais de YouTube, Cloudflare e Groq. Apenas os tokens derivados de mensagens/briefings/moodboards são filtrados por `owner_id`.

Qualquer conta autenticada pode repetir a consulta e observar a atividade agregada da instalação, inferindo uso de outros usuários. O achado não expõe conteúdo ou identidades, mas rompe o limite de isolamento esperado para uma área chamada Privacidade.

## Evidência
`backend/src/privacy/privacy.controller.ts:17-19`
```ts
@Get('usage')
usage(@CurrentUser() user: AuthenticatedUser) {
 return this.privacy.usage(user.id);
}
```

`backend/src/privacy/privacy.service.ts:32-61`
```ts
async usage(ownerId: string) {
 // Esta consulta filtra por p.owner_id = $1.
 // As três consultas abaixo não usam ownerId:
 SELECT ... FROM youtube_quota_usage WHERE usage_date = CURRENT_DATE;
 SELECT ... FROM embedding_usage_daily WHERE usage_date = CURRENT_DATE;
 SELECT ... FROM groq_usage_daily WHERE usage_date = CURRENT_DATE;
}
```

## Impacto
Vazamento de telemetria agregada entre usuários e possibilidade de inferir horários/volume de atividade. Também há risco de a interface apresentar consumo global como se fosse pessoal.

## Sugestão de correção
- Definir se cada métrica é pessoal ou operacional.
- Persistir `owner_id` nas métricas pessoais e filtrar todas as consultas por esse campo.
- Remover métricas globais da resposta comum ou movê-las para rota administrativa.
- Se houver rota administrativa, validar o privilégio no backend.

## Critérios de aceite
- [ ] Uma conta A não observa alteração em sua resposta quando somente a conta B usa YouTube, Cloudflare ou Groq.
- [ ] Todas as métricas exibidas como pessoais são filtradas por `owner_id`.
- [ ] Métricas globais, se mantidas, exigem autorização administrativa no servidor.
- [ ] Existe teste E2E com duas contas comprovando o isolamento.
--- FIM ISSUE 1 ---
```

Issue 2

```
--- ISSUE 2 ---
Título: [Segurança] Remover credenciais padrão e restringir portas de PostgreSQL e MinIO
Labels sugeridas: security, severity:high

## Descrição do problema
O ambiente Docker publica PostgreSQL, API MinIO e console MinIO em todas as interfaces do host usando credenciais previsíveis versionadas. Os mesmos valores aparecem como defaults aceitos pela configuração de desenvolvimento.

Com o Compose em execução e as portas alcançáveis, um atacante não precisa passar pela API: pode acessar diretamente o banco ou o armazenamento com privilégios amplos.

## Evidência
`backend/compose.yaml:6-11`
```yaml
POSTGRES_USER: eclipse
POSTGRES_PASSWORD: eclipse_dev
ports:
- "5432:5432"
...

`backend/compose.yaml:25-32`
```yaml
MINIO_ROOT_USER: eclipse_minio
MINIO_ROOT_PASSWORD: eclipse_minio_dev
ports:
- "9000:9000"
- "9001:9001"
...

`backend/src/config/environment.config.ts:39-43,94-103` aceita os mesmos defaults fora de produção;
`backend/evaluation/generate-report.mjs:7-13` repete o fallback do banco.

## Impacto
Comprometimento completo dos dados persistidos: leitura, alteração ou exclusão de contas, projetos, conversas, tokens de uso e arquivos de áudio. O acesso direto contorna autenticação, autorização e URLs assinadas.

## Sugestão de correção
- Remover senhas do Compose e exigir variáveis em arquivo local ignorado ou secret store.
- Gerar credenciais aleatórias no setup e recusar valores conhecidos no startup.
- Vincular portas a `127.0.0.1` por padrão (`127.0.0.1:5432:5432`, etc.).
- Usar usuário MinIO de serviço com política mínima; não usar root na aplicação.
- Remover o fallback `eclipse_dev` do script de avaliação.

## Critérios de aceite
- [ ] `docker compose config` não contém credenciais fixas provenientes do repositório.
- [ ] O startup falha para `eclipse_dev` e `eclipse_minio_dev` em qualquer ambiente compartilhado.
- [ ] PostgreSQL e MinIO escutam apenas em loopback por padrão.
- [ ] A aplicação usa credencial MinIO sem privilégios administrativos.
- [ ] Um teste/documentação de setup demonstra geração e rotação de segredos.
--- FIM ISSUE 2 ---
```